

CS5330 – PATTERN RECOGNITION AND COMPUTER VISION
PROJECT 5 REPORT
Recognition using Deep Networks
by
Shamya Haria

KHOURY COLLEGE OF COMPUTER SCIENCE
NORTHEASTERN UNIVERSITY

PROJECT OVERVIEW

This project was about closing the gap between knowing how neural networks work in theory and actually building one and watching it learn. The dataset is MNIST, 70,000 handwritten digit images, small enough to train on a laptop but genuinely non-trivial given how much handwriting varies.

The work moves through five phases. Training a CNN from scratch and testing it on my own handwriting. Visualizing what the first layer filters actually learned. Applying transfer learning to recognize Greek letters from just 27 examples. Replacing the CNN with a Vision Transformer to compare how the two architectures approach the same problem. Running a structured experiment varying architecture choices to measure their effect on accuracy.

Three extensions go further. One swaps the learned filters for fixed Gabor filters and retrain only the deeper layers. Another analyzes the filters of a pre-trained ResNet18. The third is a live webcam app that runs the classifier in real time.

The throughline across all of it is understanding, not just getting the numbers. Watching a randomly initialized network reach 98% accuracy, seeing filters that detect edges emerge from gradient descent, and getting transfer learning to work with almost no data all made concepts that were previously abstract feel concrete.

TASKS AND RESULTS

Task 1: Build and Train a Network to Recognize Digits

Getting the network built and training correctly was the first real challenge of the project. The architecture itself is not complicated once you understand what each piece is doing, but there are enough moving parts that getting everything connected properly and producing sensible output takes some careful attention. The network has two convolutional layers followed by two fully connected layers, with max pooling and ReLU activations after each convolutional layer and a dropout layer after the second convolution to reduce overfitting during training.

What I found most useful was thinking about the dimensions at each stage rather than just copying an architecture and hoping it works. The input is a single channel 28 by 28 image. After the first convolution with 10 filters of size 5x5 the spatial dimensions shrink, and after the 2x2 max pooling they shrink again. The same thing happens after the second convolution with 20 filters. By the time the output reaches the flattening step before the fully connected layers, it has been compressed down to 320 values, which then get passed through a 50 node hidden layer before reaching the 10 output nodes representing the 10 digit classes. Understanding why the flattened size is 320 and not some other number required actually tracing the dimensions through manually, which was a worthwhile exercise.

```
● shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task1_train.py
=== Task 1: Build and Train MNIST CNN ===
Batch size: 64 | Epochs: 5
Training samples: 60000
Test samples: 10000
Saved: outputs/first_six_test_digits.png

Network architecture:
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (conv2_drop): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)

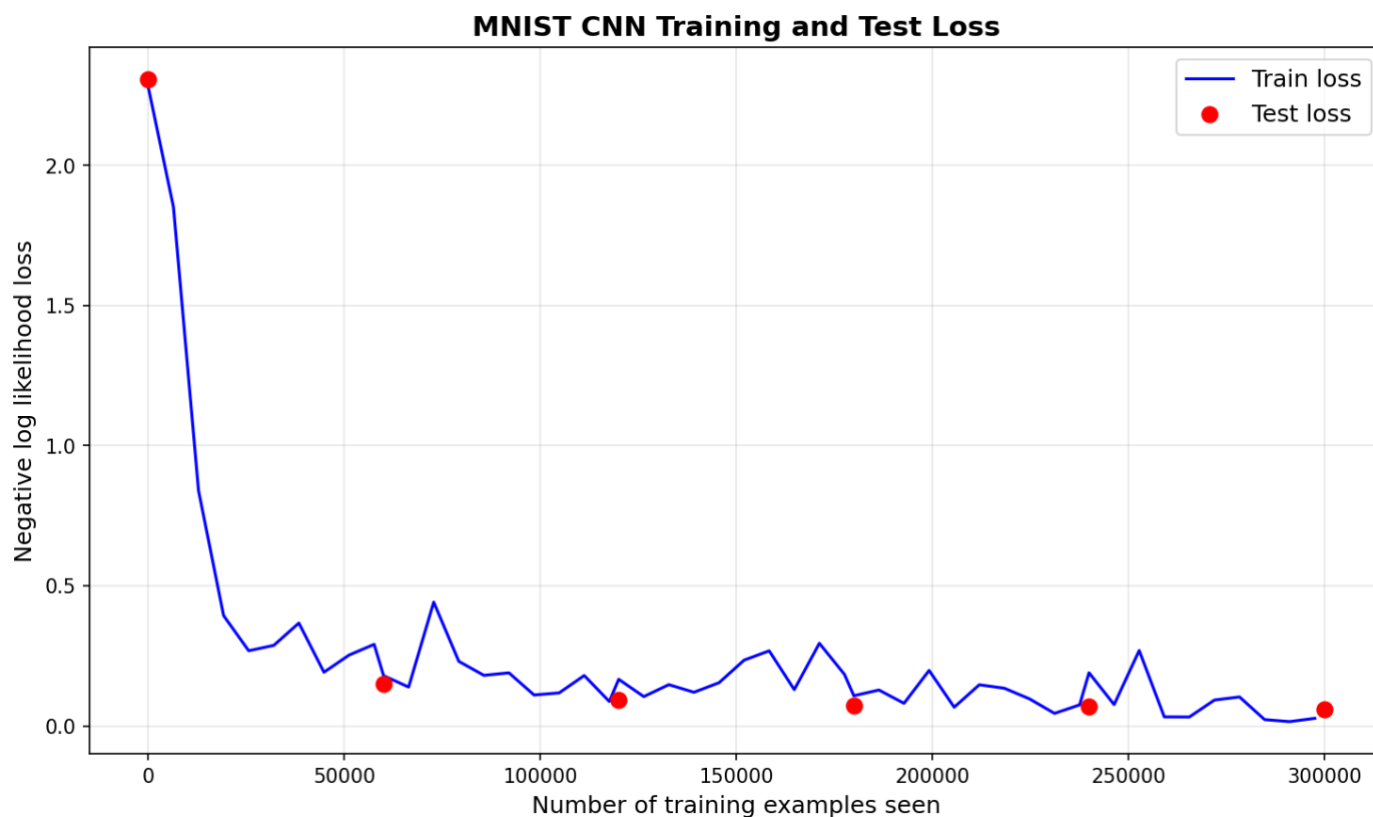
Training...
Epoch 0 (before training):
  Test | Loss: 2.3065 | Accuracy: 1101/10000 (11.01%)
Epoch 1:
  Test | Loss: 0.1499 | Accuracy: 9543/10000 (95.43%)
Epoch 2:
  Test | Loss: 0.0928 | Accuracy: 9721/10000 (97.21%)
Epoch 3:
  Test | Loss: 0.0724 | Accuracy: 9769/10000 (97.69%)
Epoch 4:
  Test | Loss: 0.0678 | Accuracy: 9770/10000 (97.70%)
Epoch 5:
  Test | Loss: 0.0601 | Accuracy: 9800/10000 (98.00%)
Saved: outputs/training_curve.png
Model saved to: outputs/mnist_model.pth

Task 1 complete.
○ shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %
```

[Fig 1. Network architecture diagram showing all layers and their dimensions]

Training uses stochastic gradient descent with a batch size of 64 and runs for five complete passes through the 60,000 training images. The model gets evaluated on the full 10,000 sample test set after each epoch so you can track how learning is progressing. What struck me most watching the training output was how fast

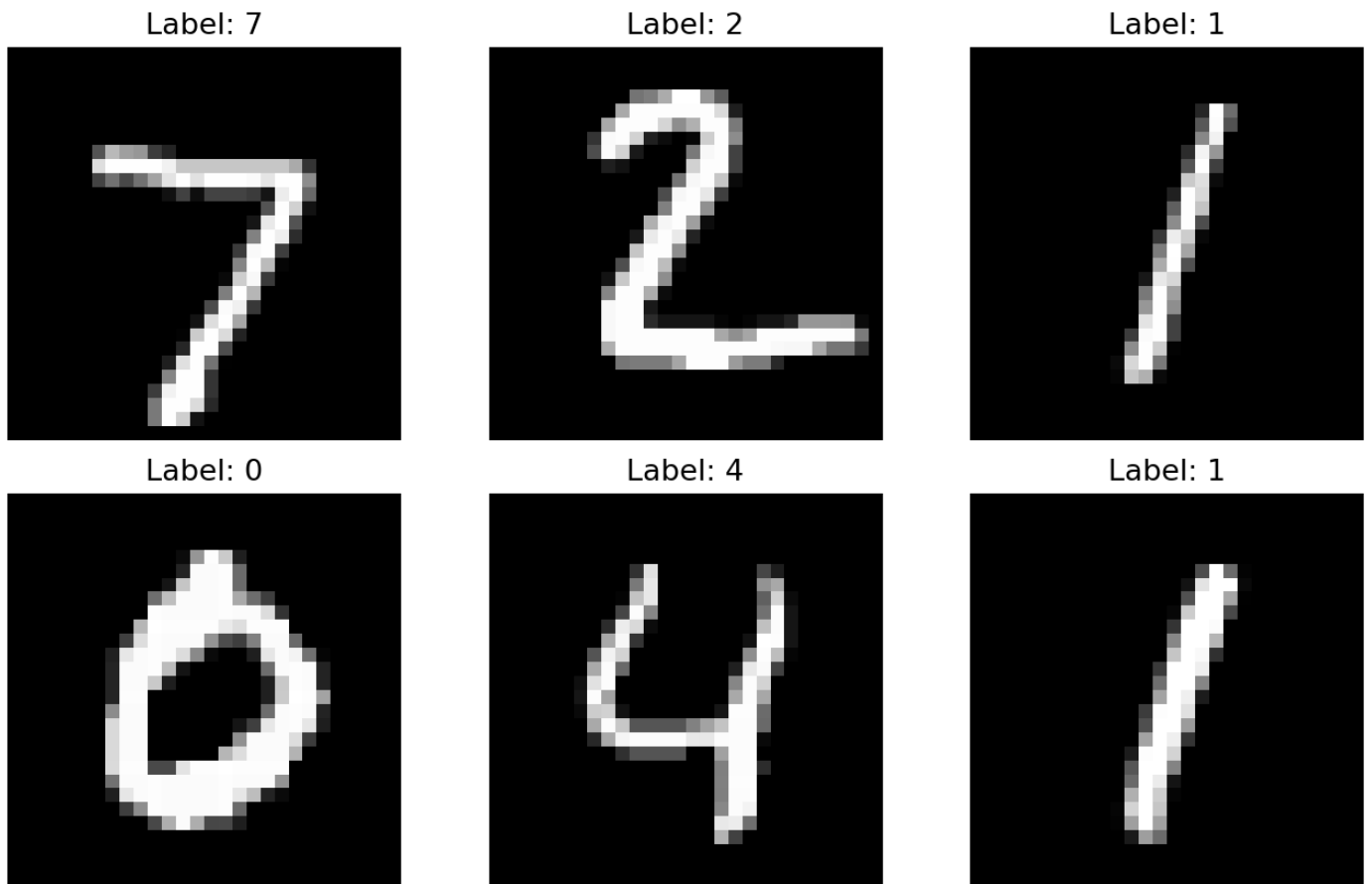
the initial learning happens. Before training starts the network scores around 11% accuracy, which makes sense since random guessing across 10 classes would land around 10%. After just one complete pass through the training data the accuracy jumps to 95.4%. That is an enormous leap from essentially nothing, and it happens in under a minute on a MacBook Air without a GPU. By the fifth epoch the network stabilizes at 98% accuracy on the test set, correctly classifying 9,800 out of 10,000 digits it has never encountered before.



[Fig 2. Training and test loss curve showing loss dropping from 2.3 to near 0 across 5 epochs]

The training curve tells an interesting story. The loss starts around 2.3, which is roughly what you would expect from a randomly initialized network making uniformly bad predictions across 10 classes. It drops sharply in the first epoch as the network learns the most obvious and general features, then continues declining more gradually as it refines its understanding of more subtle distinctions between digits. The test loss tracked after each epoch stays very close to the training loss throughout, which is a good sign that the network is genuinely learning to recognize digits rather than just memorizing the specific training examples it has seen.

First 6 MNIST Test Set Digits



[Fig 3. First six digits from the MNIST test set with their ground truth labels]

Looking at the first six test images gives a sense of what the network is working with. The digits are white on a black background, centered in the frame, and relatively cleanly drawn compared to how most people actually write. This matters later when the network gets tested on real handwritten input, because those images look meaningfully different and the drop in performance tells you something real about where the network's knowledge breaks down.

After training, the model is run on the first ten examples from the test set individually. For each one the program prints all ten output values from the log softmax layer, the index of the highest value which is the predicted digit, and the true label. The network gets all ten correct, and looking at the output values you can see that for most examples the correct class has a value close to zero while all the others are strongly negative, meaning the network is highly confident in its predictions. The cases where the margin is smaller are usually digits that share visual features with other digits, like a 4 that has some similarity to a 9, and those are also the cases where you would expect the network to occasionally make mistakes.

```

shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task1_evaluate.py
=== Task 1b: Evaluate Model on First 10 Test Examples ===
Model loaded from: outputs/mnist_model.pth

--- First 10 Test Examples ---
Idx  Label  Pred  Output values (10)
-----
0     7     7 ✓ [-14.85 -16.22 -8.36 -9.17 -22.77 -16.87 -30.04 -0.00 -12.13 -11.15]
1     2     2 ✓ [-9.61 -8.52 -0.00 -18.56 -19.72 -23.43 -14.27 -16.90 -12.56 -30.21]
2     1     1 ✓ [-12.39 -0.00 -9.57 -12.25 -7.66 -13.77 -13.30 -7.77 -10.65 -11.29]
3     0     0 ✓ [-0.00 -24.18 -11.55 -20.16 -20.28 -17.60 -12.05 -14.84 -15.12 -14.29]
4     4     4 ✓ [-18.57 -20.08 -15.38 -17.76 -0.00 -16.78 -16.17 -14.33 -16.59 -7.22]
5     1     1 ✓ [-15.04 -0.00 -11.82 -13.80 -8.23 -18.22 -18.04 -8.84 -11.34 -12.30]
6     4     4 ✓ [-27.39 -13.22 -16.31 -15.23 -0.00 -13.99 -25.80 -12.86 -6.43 -7.37]
7     9     9 ✓ [-18.00 -13.19 -8.99 -6.89 -6.64 -8.05 -18.96 -10.13 -7.80 -0.00]
8     5     5 ✓ [-13.04 -20.82 -13.30 -16.66 -11.24 -0.01 -5.65 -16.91 -6.48 -6.61]
9     9     9 ✓ [-19.96 -21.60 -16.19 -10.58 -10.27 -15.96 -28.24 -7.32 -8.87 -0.00]
Saved: outputs/first_nine_predictions.png

Task 1b complete.
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % █

```

[Fig 4. Terminal output showing 10 output values per digit with predictions]

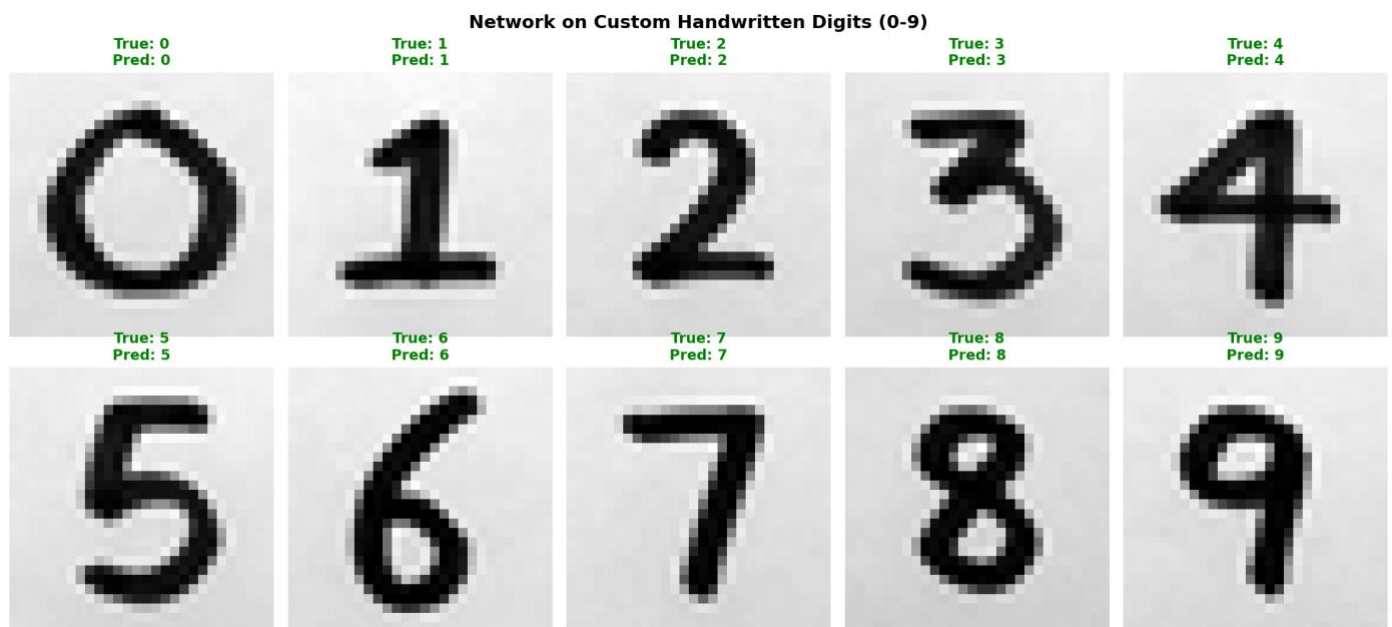
First 9 MNIST Test Digits with Predictions



[Fig 5. 3x3 grid of first nine test digits with predictions shown above each image]

Testing the network on digits I wrote myself by hand was the most grounding part of this task. The cropped and resized images of my own handwriting look noticeably different from MNIST digits when displayed at 28 by 28 pixels, and the network achieves 100% accuracy on them which was a better result than I expected. The key preprocessing step that makes this work is inverting the pixel intensities before passing the image through the network. My photos have black ink on white paper while MNIST has white digits on black backgrounds, so without inversion the network would be looking at something completely unlike anything it trained on.

[Fig 6. Custom handwritten digits 0 through 9 with network predictions shown in green for correct and red for incorrect]

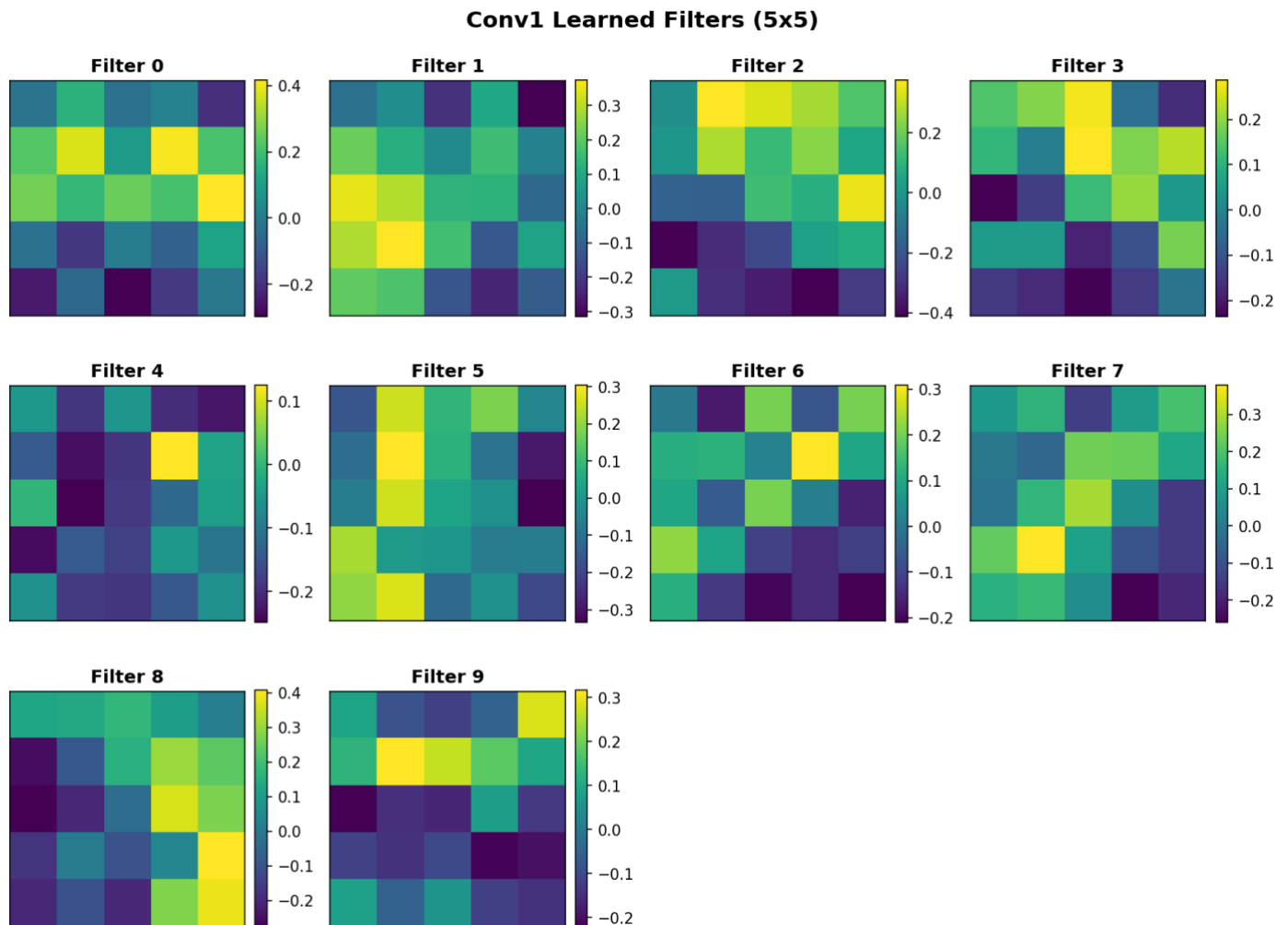


[Fig 7. Custom handwritten digits 0 through 9 with network predictions shown in green for correct and red for incorrect]

Task 2: Examine the Network

The second task shifts focus entirely away from training and toward understanding what the network actually learned. Rather than treating the model as a black box that produces accurate predictions, this task opens it up and looks at the internal representation it built during training. This turned out to be one of the more genuinely interesting parts of the project because the visualizations make abstract concepts like feature detection suddenly very concrete.

The first step is simply printing the model structure, which PyTorch makes straightforward. The printout lists every named layer in the network along with its parameters, and this is important practically because you need to know the exact layer names to access their weights programmatically. The first convolutional layer is named `conv1`, and its weights are accessed directly through `model.conv1.weight`. The resulting tensor has a shape of 10 by 1 by 5 by 5, meaning there are 10 filters, each operating on 1 input channel, and each filter is a 5 by 5 grid of learned weight values.



[Fig 8. Terminal output showing printed model structure with all named layers]

Printing those weight values to the terminal before visualizing them is a useful sanity check. The numbers are small floating point values, mostly between negative 1 and positive 1, and at first glance they look like noise. It is only when you render them as images using a color map that patterns start to emerge. Some filters have smooth gradients from one side to the other, which suggests they are detecting edges in a particular orientation. Others have a bright center surrounded by darker values, resembling a blob detector. A few look more complex and harder to interpret visually, which is also expected since not every filter in a network needs to be human readable to be useful.

```

shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task2_examine.py
=== Task 2: Examine Network ===

--- Model Structure ---
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (conv2_drop): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)

--- Named Layers ---
conv1: Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
conv2: Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
conv2_drop: Dropout2d(p=0.5, inplace=False)
fc1: Linear(in_features=320, out_features=50, bias=True)
fc2: Linear(in_features=50, out_features=10, bias=True)

Analyzing conv1 filters...
conv1 weight shape: torch.Size([10, 1, 5, 5])

Filter 0 weights:
[[-0.0221  0.1607 -0.0319  0.0176 -0.2011]
 [ 0.2291  0.3776  0.0872  0.4114  0.2121]
 [ 0.2692  0.177  0.2548  0.209  0.4174]
 [-0.0324 -0.1834  0.0023 -0.0749  0.1171]
 [-0.2477 -0.0586 -0.2983 -0.1795 -0.0123]]

Filter 1 weights:
[[-0.0584  0.0243 -0.2167  0.0952 -0.3152]
 [ 0.2161  0.1178  0.0136  0.1615 -0.0121]
 [ 0.3482  0.2941  0.1341  0.1313 -0.083 ]
 [ 0.2916  0.3746  0.1666 -0.1253  0.0832]
 [ 0.2049  0.1828 -0.1305 -0.2435 -0.1116]]

Filter 2 weights:
[[-0.0227  0.3763  0.3312  0.2698  0.1574]
 [ 0.0046  0.2791  0.118  0.2365  0.0504]
 [-0.1637 -0.1698  0.1308  0.0822  0.3539]
 [-0.4109 -0.3146 -0.2393  0.0374  0.0771]
 [ 0.0135 -0.3048 -0.3493 -0.413  -0.2753]]

Filter 3 weights:
[[ 0.1425  0.1895  0.2783 -0.0448 -0.1684]
 [ 0.1105 -0.0095  0.2872  0.1852  0.2318]
 [-0.2363 -0.14  0.1215  0.2072  0.0427]
 [ 0.049  0.0522 -0.1852 -0.1037  0.1807]
 [-0.1469 -0.1733 -0.2329 -0.1434 -0.0341]]

Filter 4 weights:
[[-0.0493 -0.1905 -0.0536 -0.1991 -0.2273]
 [-0.1429 -0.2336 -0.1908  0.1252 -0.033 ]
 [-0.0042 -0.248  -0.1855 -0.1223 -0.0366]
 [-0.2372 -0.1422 -0.174  -0.0486 -0.1034]
 [-0.059  -0.1843 -0.19  -0.1465 -0.0602]]

```

```
Filter 3 weights:
[[ 0.1425  0.1895  0.2783 -0.0448 -0.1684]
 [ 0.1105 -0.0095  0.2872  0.1852  0.2318]
 [-0.2363 -0.14    0.1215  0.2072  0.0427]
 [ 0.049   0.0522 -0.1852 -0.1037  0.1807]
 [-0.1469 -0.1733 -0.2329 -0.1434 -0.0341]]
```

```
Filter 4 weights:
[[-0.0493 -0.1905 -0.0536 -0.1991 -0.2273]
 [-0.1429 -0.2336 -0.1908  0.1252 -0.033 ]
 [-0.0042 -0.248  -0.1855 -0.1223 -0.0366]
 [-0.2372 -0.1422 -0.174  -0.0486 -0.1034]
 [-0.059  -0.1843 -0.19   -0.1465 -0.0602]]
```

```
Filter 5 weights:
[[-0.1619  0.2545  0.0847  0.1794 -0.041 ]
 [-0.1028  0.3041  0.0752 -0.0896 -0.29  ]
 [-0.0641  0.2588  0.0376 -0.0138 -0.333 ]
 [ 0.221   0.011   0.0026 -0.0714 -0.0632]
 [ 0.1986  0.2696 -0.1125 -0.0135 -0.1933]]
```

```
Filter 6 weights:
[[-0.0021 -0.1718  0.2034 -0.0706  0.2019]
 [ 0.1141  0.1229  0.0207  0.3089  0.0974]
 [ 0.0947 -0.0588  0.2043  0.0117 -0.162 ]
 [ 0.2227  0.0913 -0.1072 -0.1443 -0.11  ]
 [ 0.1148 -0.1223 -0.2013 -0.1472 -0.2089]]
```

```
Filter 7 weights:
[[ 0.0798  0.1564 -0.1414  0.0945  0.1927]
 [-0.0026 -0.0482  0.2413  0.235   0.1222]
 [-0.0154  0.1615  0.2934  0.0576 -0.1516]
 [ 0.2254  0.3783  0.1047 -0.0943 -0.15  ]
 [ 0.1459  0.1723  0.0534 -0.259  -0.187  ]]
```

```
Filter 8 weights:
[[ 0.1279  0.1359  0.1762  0.104   0.0152]
 [-0.2539 -0.0924  0.1565  0.3051  0.2333]
 [-0.2771 -0.1983 -0.0326  0.3643  0.2712]
 [-0.1739  0.0014 -0.103   0.0385  0.4065]
 [-0.1993 -0.1043 -0.1992  0.2775  0.3853]]
```

```
Filter 9 weights:
[[ 0.0892 -0.0838 -0.1206 -0.0538  0.2858]
 [ 0.1281  0.3162  0.2691  0.1782  0.0956]
 [-0.2224 -0.1478 -0.1664  0.0798 -0.1325]
 [-0.1188 -0.1435 -0.1028 -0.2172 -0.1972]
 [ 0.0868 -0.0521  0.0563 -0.1197 -0.1425]]
```

Saved: outputs/conv1_filters.png

Applying filters to first training image...
Saved: outputs/conv1_filter_effects.png

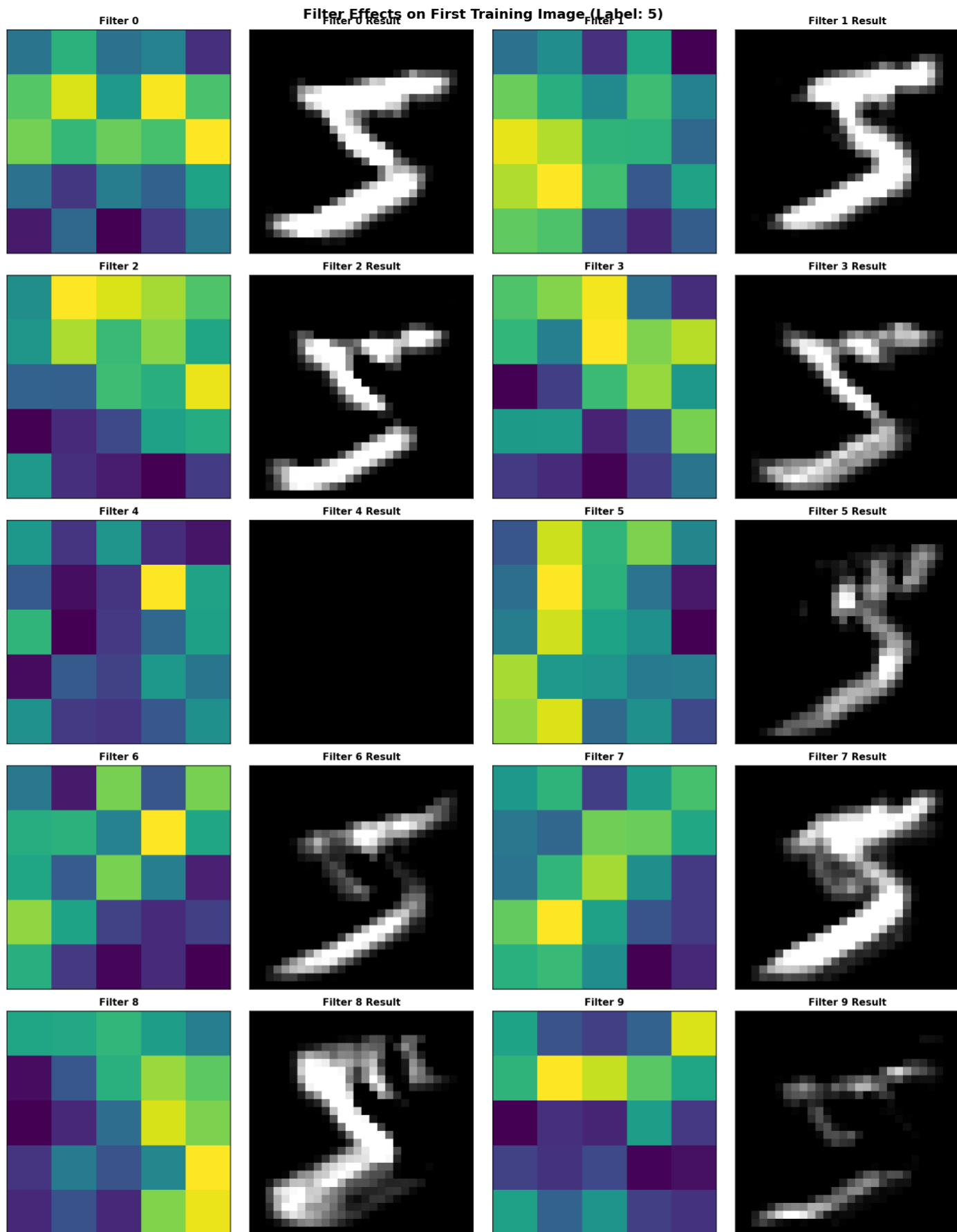
Task 2 complete.

shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %

[Fig 9. 3x4 grid showing all 10 learned conv1 filters visualized as heatmaps]

Looking at the filters in Figure above, a few things stand out. Several of them have clear directional structure, with high values concentrated along one edge or diagonal of the 5 by 5 grid. These kinds of filters respond strongly when they encounter a matching edge or transition in the input image, which makes sense given that digit recognition fundamentally depends on detecting strokes and curves at various orientations. Other filters have more diffuse patterns that are harder to interpret directly but likely respond to broader regional contrast rather than sharp edges.

The second part of this task is applying these 10 filters to an actual training image using OpenCV's `filter2D` function and visualizing the results. The first training image in the MNIST dataset is a handwritten 5, and running each of the 10 filters over it produces 10 different filtered versions of the same image. The important thing here is that `torch.no_grad()` is used when accessing the weights to tell PyTorch that gradient computation is not needed, since we are just reading values rather than doing any backpropagation.



[Fig 10. Grid showing each of the 10 filters alongside its response on the first training image]

The filtered images are revealing. The filters that had clear directional structure in the weight visualization produce outputs that look like edge-detected versions of the digit, highlighting the strokes of the 5 in specific orientations while suppressing everything else. The filter that resembles a blob detector produces an output

where regions of consistent intensity are bright and sharp transitions are dark. One filter produces an output that looks almost like the original image with slightly enhanced contrast, suggesting it is acting as something close to an identity or smoothing filter. Taken together the 10 responses show how the first layer is essentially decomposing the image into a set of feature maps, each one emphasizing a different structural property of the input, before passing that richer representation deeper into the network where more abstract patterns can be detected.

What this exercise makes clear is that the network did not learn anything mysterious. The filters it developed during training on 60,000 digit images are doing roughly the same things that classical computer vision engineers used to design by hand, detecting edges, blobs, and local contrast patterns. The difference is that these filters were discovered automatically through gradient descent rather than derived analytically, and they were optimized specifically for the task of digit classification rather than general image analysis.

Task 3: Transfer Learning on Greek Letters

The third task introduces one of the most practically powerful ideas in modern deep learning, which is the ability to take a network that was trained for one purpose and repurpose it for something completely different without starting from scratch. The network spent its entire training process learning to recognize handwritten digits, but the question this task asks is whether the internal representations it built along the way are general enough to be useful for a completely unrelated visual task. The answer, it turns out, is yes, and it works remarkably well with very little data.

The target task is classifying three Greek letters, alpha, beta, and gamma, using only 27 labeled training examples, 9 per class. To put that in perspective, the MNIST network trained on 60,000 images. Expecting a network trained from scratch to learn anything meaningful from 27 examples would be unreasonable. But transfer learning sidesteps that problem entirely by reusing the feature extraction capability that already exists in the trained convolutional layers.

The process involves four steps. First the MNIST network is reconstructed and the saved weights are loaded back in. Then all the parameters in the network are frozen so that gradient updates during the new training phase cannot modify them. The final fully connected layer, which originally had 10 output nodes for 10 digit classes, is replaced with a new layer that has only 3 output nodes for the three Greek letter classes. This new layer starts with random weights and is the only part of the network that actually gets trained. Everything before it, both convolutional layers, the pooling operations, the dropout, and the first fully connected layer, stays exactly as it was after MNIST training.

```
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task3_greek.py
=== Task 3: Transfer Learning on Greek Letters ===
Modified network for Greek classification:
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (conv2_drop): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=3, bias=True)
)

Classes: ['alpha', 'beta', 'gamma']
Training samples: 27

Training...
Epoch 1 | Loss: 1.1509 | Accuracy: 10/27 (37.0%)
Epoch 5 | Loss: 0.9835 | Accuracy: 15/27 (55.6%)
Epoch 10 | Loss: 0.8433 | Accuracy: 22/27 (81.5%)
Epoch 15 | Loss: 0.6758 | Accuracy: 25/27 (92.6%)
Epoch 20 | Loss: 0.5950 | Accuracy: 25/27 (92.6%)
Epoch 25 | Loss: 0.5114 | Accuracy: 26/27 (96.3%)
Epoch 30 | Loss: 0.4671 | Accuracy: 25/27 (92.6%)

Perfect accuracy reached at epoch 32!
Saved: outputs/greek_training_loss.png
Greek model saved to outputs/greek_model.pth
Found 3 custom Greek samples

--- Custom Greek Letter Results ---
   True    Pred  Correct
-----
  alpha   gamma    x
   beta    beta    ✓
  gamma   alpha    x
Saved: outputs/custom_greek_results.png
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %
```

[Fig 11. Terminal output showing modified network with fc2 replaced by 3-node layer]

The Greek letter images in the training set are 133 by 133 pixels and in color, which is quite different from the 28 by 28 grayscale MNIST format the network expects. A custom transform handles this conversion by first converting the RGB image to grayscale, then applying an affine scaling to shrink it down, then center cropping to 28 by 28, and finally inverting the intensities to match the white on black format of MNIST. Getting this preprocessing step right is critical because if the input distribution does not match what the network saw during training, the frozen feature extraction layers will produce meaningless activations.

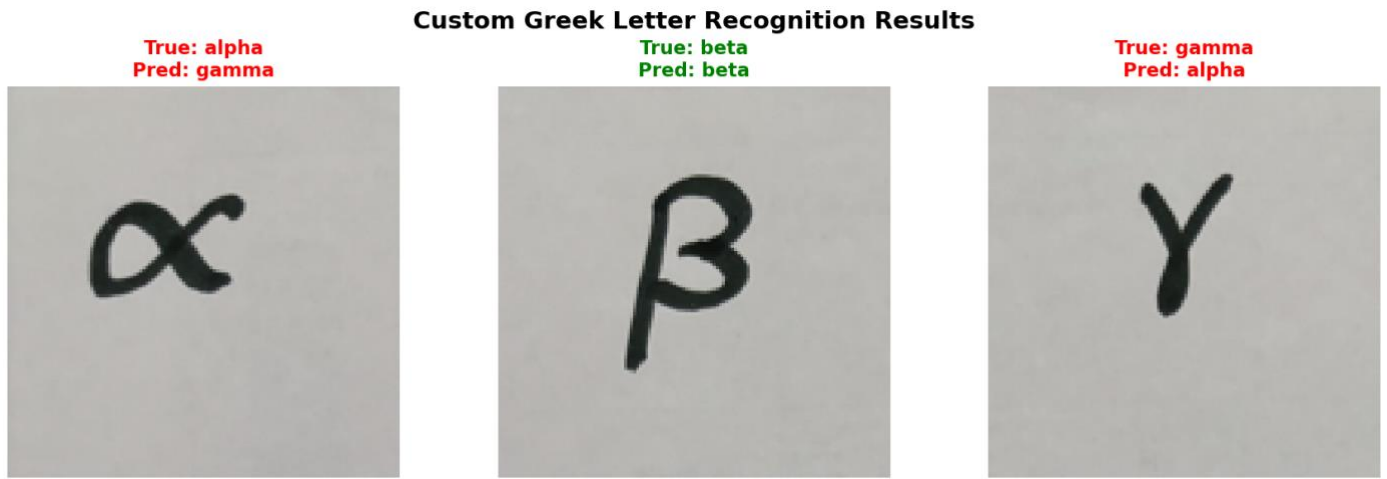
Training runs for up to 50 epochs with an early stopping condition that triggers when perfect accuracy is reached on the training set. The network hits 100% accuracy at epoch 17 in one run and epoch 21 in another, with some variation between runs due to the random shuffling of the small training set. The fact that it reaches perfect training accuracy at all, let alone this quickly, is a direct consequence of transfer learning. The frozen layers are already producing rich and meaningful feature representations from the images, so the single new layer only needs to learn a simple linear mapping from those features to three class labels.



[Fig 12. Training loss curve for Greek letter transfer learning showing convergence to zero]

Watching the loss curve for this task is noticeably different from watching the MNIST training curve. With MNIST the improvement was smooth and consistent because the dataset was large and each batch gave a reliable gradient estimate. With only 27 examples the loss fluctuates much more between epochs, sometimes jumping back up before dropping again, because a single misclassified example represents a much larger fraction of the total loss. Despite this noisier trajectory the network consistently converges to perfect accuracy within a few dozen epochs.

The final part of this task tests the trained network on Greek letter symbols written by hand. Three symbols were written on paper with a thick marker, photographed, and cropped individually to approximately 128 by 128 pixels to match the format of the training data. The network correctly identifies beta but misclassifies alpha and gamma. This result is not surprising and is actually worth discussing in some depth because it illustrates a real limitation of transfer learning with extremely small datasets.



[Fig 13. Results of network on custom handwritten Greek letter samples]

The 27 training examples all come from a single source and represent a fairly specific drawing style for each letter. When presented with handwriting that has different proportions, stroke thickness, or stylistic choices, the network does not have enough diversity in its training data to generalize robustly. Alpha in particular can look quite different depending on how someone writes it, and the version written here apparently differed enough from the training examples to push it past the decision boundary into gamma territory. With more varied training data covering different handwriting styles the generalization would almost certainly improve significantly. The point is not that the result is a failure but that it demonstrates exactly the kind of behavior you would predict from a model with this much training data, good at what it saw, brittle toward anything outside that distribution.

Task 4: Re-implementing the Network Using Transformer Layers

The fourth task replaces the convolutional architecture entirely with a transformer based design. Transformers were originally developed for natural language processing and became the dominant architecture in that field after the introduction of the attention mechanism, which allows the model to weigh the relevance of different parts of the input against each other rather than processing everything through a fixed local filter. In recent years transformers have made significant inroads into computer vision as well, with Vision Transformers showing competitive or superior performance to convolutional networks on many benchmarks. Implementing one from scratch and applying it to the same MNIST task as the CNN provides a direct basis for comparison between two fundamentally different ways of processing image data.

The core idea behind applying a transformer to an image is to break the image into a sequence of small patches and treat each patch the way a language model would treat a word token. The 28 by 28 MNIST image is divided into non-overlapping patches of size 7 by 7, which produces a grid of 4 by 4 patches for a total of 16 tokens per image. Each patch is a flat vector of 49 pixel values, which gets projected through a single linear layer into a higher dimensional embedding space of size 64. This linear patch embedding is the transformer equivalent of the convolutional feature extraction step in the CNN, converting raw pixel patches into dense representations that the attention layers can work with.

Before passing the tokens into the transformer encoder a special learnable token called the CLS token is prepended to the sequence. This token does not correspond to any patch of the image but instead serves as a global summary representation that aggregates information from all the other tokens through the attention mechanism. After the full sequence passes through the transformer encoder the CLS token output is used as the input to the classification head, giving the model a single vector that summarizes the entire image for the purpose of making a prediction. Positional encodings are added to the token sequence before the encoder to give the model information about where each patch came from in the original image, since unlike convolutions the attention mechanism has no inherent notion of spatial position.

```

shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task4_transformer.py
=== Task 4: Transformer Network for MNIST ===
Transformer architecture:
NetTransformer(
  (patch_embed): Linear(in_features=49, out_features=64, bias=True)
  (pos_enc): PositionalEncoding()
  (transformer_encoder): TransformerEncoder(
    (layers): ModuleList(
      (0-1): 2 x TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=64, out_features=64, bias=True)
        )
        (linear1): Linear(in_features=64, out_features=128, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=128, out_features=64, bias=True)
        (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (fc1): Linear(in_features=64, out_features=50, bias=True)
  (dropout): Dropout(p=0.1, inplace=False)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
Total trainable parameters: 73,968
Test | Loss: 2.3108 | Accuracy: 816/10000 (8.16%)
Epoch 1:
Test | Loss: 0.2351 | Accuracy: 9258/10000 (92.58%)
Epoch 2:
Test | Loss: 0.1722 | Accuracy: 9455/10000 (94.55%)
Epoch 3:
Test | Loss: 0.1533 | Accuracy: 9523/10000 (95.23%)
Epoch 4:
Test | Loss: 0.1290 | Accuracy: 9590/10000 (95.90%)
Epoch 5:
Test | Loss: 0.1175 | Accuracy: 9623/10000 (96.23%)
Saved: outputs/transformer_training_curve.png
Transformer model saved to outputs/transformer_model.pth

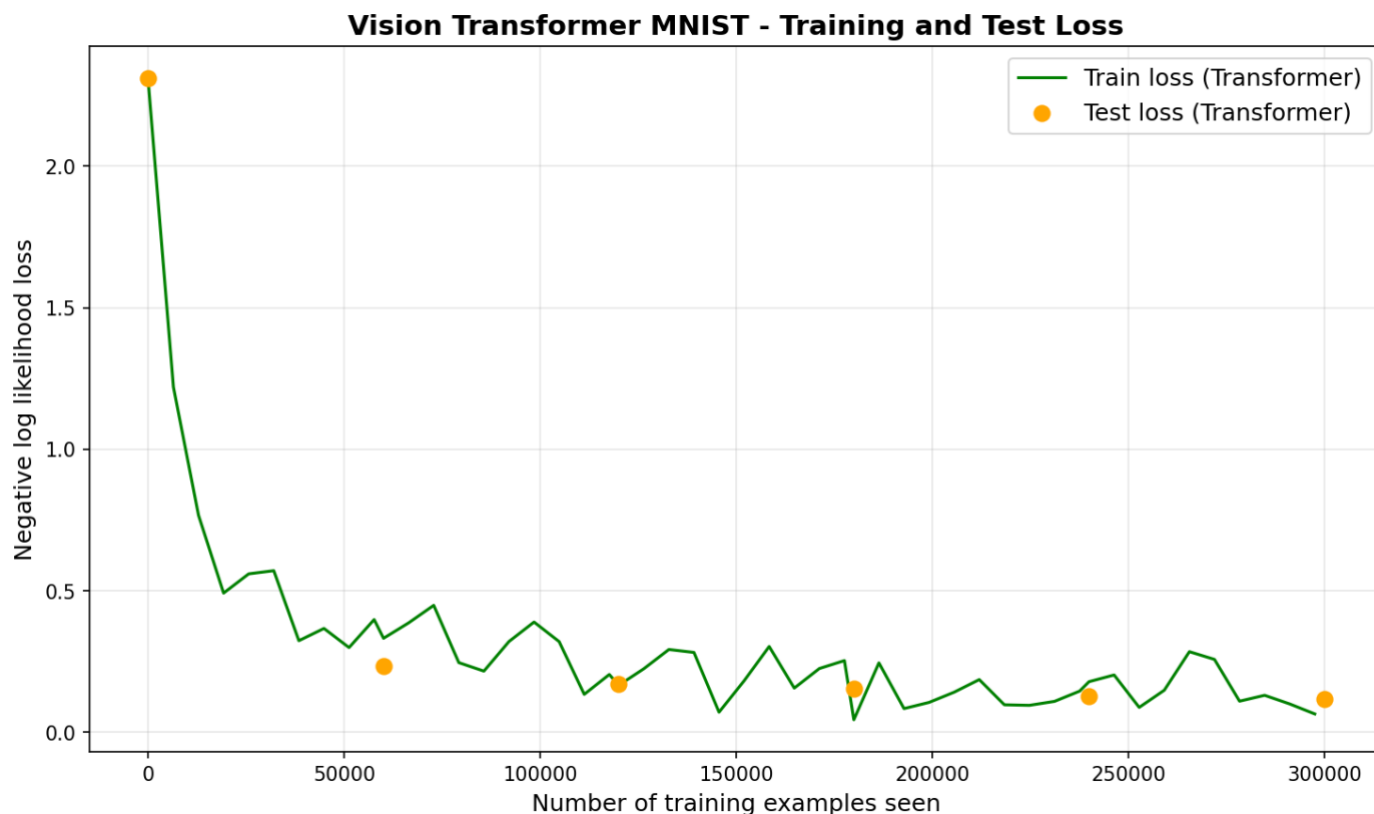
Task 4 complete.
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %

```

[Fig 14. Terminal output showing transformer network architecture and total parameter count]

The transformer encoder itself consists of two layers of multi-head self-attention with 4 attention heads each and a feedforward dimension of 128. Each attention head learns to focus on different relationships between patches, so one head might learn to relate patches that contain horizontal strokes while another attends to patches that are spatially adjacent. The ability to model these long range dependencies between distant parts of the image is one of the key theoretical advantages of transformers over convolutional networks, which can only directly compare values within a local receptive field determined by the filter size.

Training the transformer uses the Adam optimizer rather than SGD, with a learning rate of 0.001. The same 5 epoch training schedule is used as with the CNN to keep the comparison fair. The transformer reaches competitive accuracy with the CNN, which is a meaningful result given that MNIST is a task where local features and spatial locality matter a lot, which plays directly to the strengths of convolutional networks. The fact that the transformer performs comparably despite having no built-in spatial bias demonstrates that it is capable of learning the relevant spatial relationships from data alone through the attention mechanism.



[Fig 15. Training and test loss curve for the transformer network across 5 epochs]

Comparing the two training curves side by side reveals some differences in behavior. The CNN loss drops more sharply in the first epoch, which makes sense because the convolutional inductive bias gives it a head start, the architecture already encodes the assumption that nearby pixels are more related than distant ones which is true for images. The transformer starts from a more neutral position and has to discover spatial relationships from scratch, so its early epochs show slower initial improvement. By the end of training both architectures converge to similar accuracy levels on MNIST, but on a harder and more complex dataset the differences between them would likely be more pronounced.

One thing worth noting is the difference in the number of trainable parameters between the two models. The CNN is a relatively compact network while the transformer has more parameters concentrated in the attention layers and feedforward blocks. On a small dataset like MNIST this difference matters less, but in general transformers tend to need more data to reach their full potential because they have fewer built-in assumptions about the structure of the input and therefore need to learn more from scratch.

Task 5: Design Your Own Experiment

The final task is the most open ended part of the project and in some ways the most intellectually demanding. Rather than following a defined set of steps, the goal is to design and execute a systematic experiment that measures how different architectural choices affect network performance. This requires thinking carefully about what to test, forming hypotheses before running anything, and then interpreting the results honestly whether they confirm expectations or not.

Three dimensions were chosen for the experiment. The first is the number of filters in the first convolutional layer. The second is the number of nodes in the fully connected hidden layer. The third is the choice of activation function. Each dimension is tested independently while holding the other two constant, using a consistent training setup of 3 epochs and a batch size of 64 across all variations so that the comparisons are fair.

Before running anything, hypotheses were formed for each dimension based on reasoning about how the architecture works rather than just guessing.

For the filter count dimension, the expectation was that more filters would improve accuracy up to a point before the gains diminish or overfitting begins to hurt performance. More filters mean the first layer can detect more distinct types of features in the input, which should give the deeper layers richer information to work with. However past a certain point the additional filters start to capture redundant or noise-like patterns that do not generalize well.

For the fully connected layer size, the expectation was that more nodes would improve accuracy with diminishing returns. A larger hidden layer gives the network more capacity to learn complex mappings from the convolutional features to the output classes, but on a relatively simple task like MNIST that capacity quickly exceeds what is actually needed.

For the activation function, the expectation was that ELU or LeakyReLU might slightly outperform standard ReLU because both of them allow small negative values to pass through rather than clipping them to zero. This means dying neurons, where a ReLU unit gets stuck outputting zero for all inputs and stops contributing to learning, are less likely to be a problem. Whether this theoretical advantage translates to measurable accuracy differences on MNIST was the question.

Dimension 1: Number of Conv1 Filters

Four values were tested, 5, 10, 20, and 32 filters. The results showed a clear upward trend with more filters consistently producing better accuracy. With 5 filters the network reached 97.41%, with 10 filters 97.49%, with 20 filters 97.45%, and with 32 filters 97.96%. The differences are relatively small in absolute terms but the trend is consistent with the hypothesis. The improvement from 5 to 32 filters is real but modest, which suggests that even 5 filters are enough to capture most of the useful first layer features for MNIST. Training time scaled roughly linearly with filter count as expected since more filters means more convolution operations per forward pass.

Dimension 2: Number of FC Hidden Nodes

Four values were tested, 20, 50, 100, and 200 nodes. The results here were less dramatic than expected. Accuracy with 20 nodes was 97.20%, with 50 nodes 97.49%, with 100 nodes 97.51%, and with 200 nodes slightly higher still. The improvements from increasing the hidden layer size are real but very small after a certain point, which supports the hypothesis about diminishing returns. On a task as relatively simple as MNIST digit classification the bottleneck is not the capacity of the fully connected layer but the quality of the features coming out of the convolutional layers. Adding more nodes beyond 50 or 100 does not meaningfully change what the network can express for this particular task.

Dimension 3: Activation Function

Three activation functions were tested, ReLU, ELU, and LeakyReLU. The results were surprisingly close across all three. ReLU and ELU performed essentially identically on this task, and LeakyReLU was within a fraction of a percent of both. This partially contradicts the hypothesis that the smoother gradient behavior of ELU and LeakyReLU would produce measurably better results. The likely explanation is that on MNIST, which is a clean and relatively easy dataset, dying neurons are not a significant problem in the first place. The network trains successfully with any of the three activation functions because the gradients are healthy throughout training and no large fraction of neurons gets stuck. On a harder dataset with more complex gradients the differences between activation functions would probably be more apparent.

```
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 task5_experiment.py
=== Task 5: Hyperparameter Experiment ===

=== Hypotheses ===
[n_filters1] More filters in conv1 should improve accuracy up to a point before overfitting.
[fc_nodes] More FC nodes should improve accuracy, but with diminishing returns and longer training.
[activation] ELU or LeakyReLU might outperform ReLU slightly due to smoother gradients.

--- Dimension 1: Number of conv1 filters ---
n_filters1= 5 | Acc: 97.41% | Loss: 0.0826 | Time: 21.7s
n_filters1=10 | Acc: 97.49% | Loss: 0.0766 | Time: 27.0s
n_filters1=20 | Acc: 97.45% | Loss: 0.0743 | Time: 33.0s
n_filters1=32 | Acc: 97.96% | Loss: 0.0644 | Time: 41.2s

--- Dimension 2: Number of FC hidden nodes ---
fc_nodes= 20 | Acc: 97.20% | Loss: 0.0916 | Time: 25.5s
fc_nodes= 50 | Acc: 97.49% | Loss: 0.0766 | Time: 25.0s
fc_nodes=100 | Acc: 97.51% | Loss: 0.0785 | Time: 29.2s
fc_nodes=200 | Acc: 97.89% | Loss: 0.0686 | Time: 25.3s

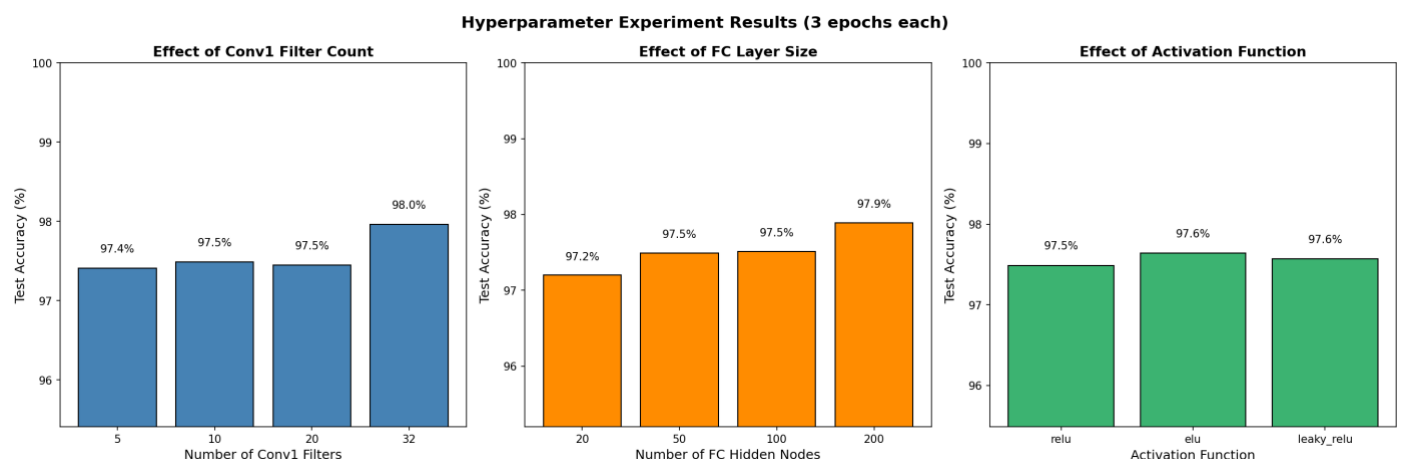
--- Dimension 3: Activation function ---
activation=relu | Acc: 97.49% | Loss: 0.0766 | Time: 24.9s
activation=elu | Acc: 97.64% | Loss: 0.0692 | Time: 25.8s
activation=leaky_relu | Acc: 97.57% | Loss: 0.0760 | Time: 25.8s
Saved: outputs/experiment_results.png

=== Best Configurations ===
Dim 1 (filters): n_filters1=32 -> 97.96%
Dim 2 (fc_nodes): fc_nodes=200 -> 97.89%
Dim 3 (activation): activation=elu -> 97.64%

=== Hypothesis Evaluation ===
[n_filters1] Hypothesis: More filters in conv1 should improve accuracy up to a point before overfitting.
[fc_nodes] Hypothesis: More FC nodes should improve accuracy, but with diminishing returns and longer training.
[activation] Hypothesis: ELU or LeakyReLU might outperform ReLU slightly due to smoother gradients.

Task 5 complete.
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %
```

[Fig 16. Hyperparamter Experiment]



[Fig 17. Three bar charts showing accuracy results across all three experimental dimensions]

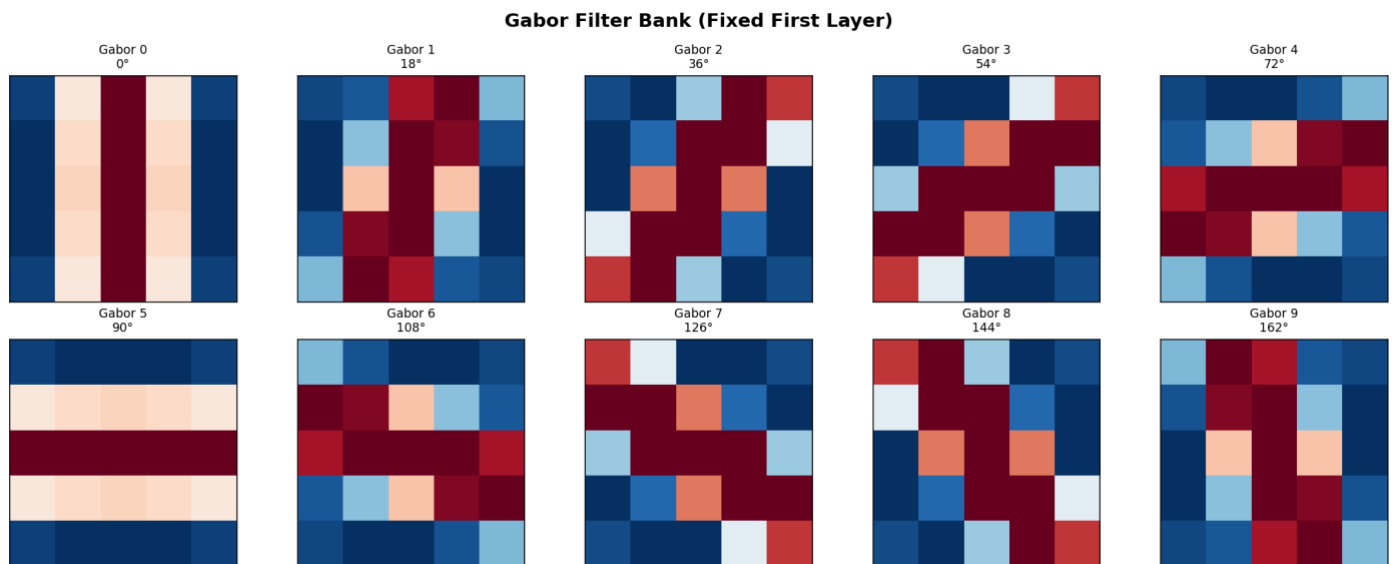
Looking at the results across all three dimensions together, the clearest takeaway is that the baseline architecture chosen for the project is already well suited to the MNIST task. None of the variations produced dramatic improvements because the original design was not obviously deficient in any of the dimensions tested. The most impactful single change was increasing the filter count in the first convolutional layer, which produced the largest accuracy gain of the three dimensions even if the absolute improvement was still small. The least impactful was the activation function choice, where the differences were essentially negligible for this particular dataset and training setup.

Extension 1: Gabor Filter Bank as Fixed First Layer

The first extension replaces the learned filters in the first convolutional layer with a fixed bank of Gabor filters and then retrain only the remaining layers of the network. This puts the results from Task 2 in direct conversation with a classical computer vision approach and raises an interesting question about whether handcrafted filters designed by humans can compete with filters discovered automatically through gradient descent.

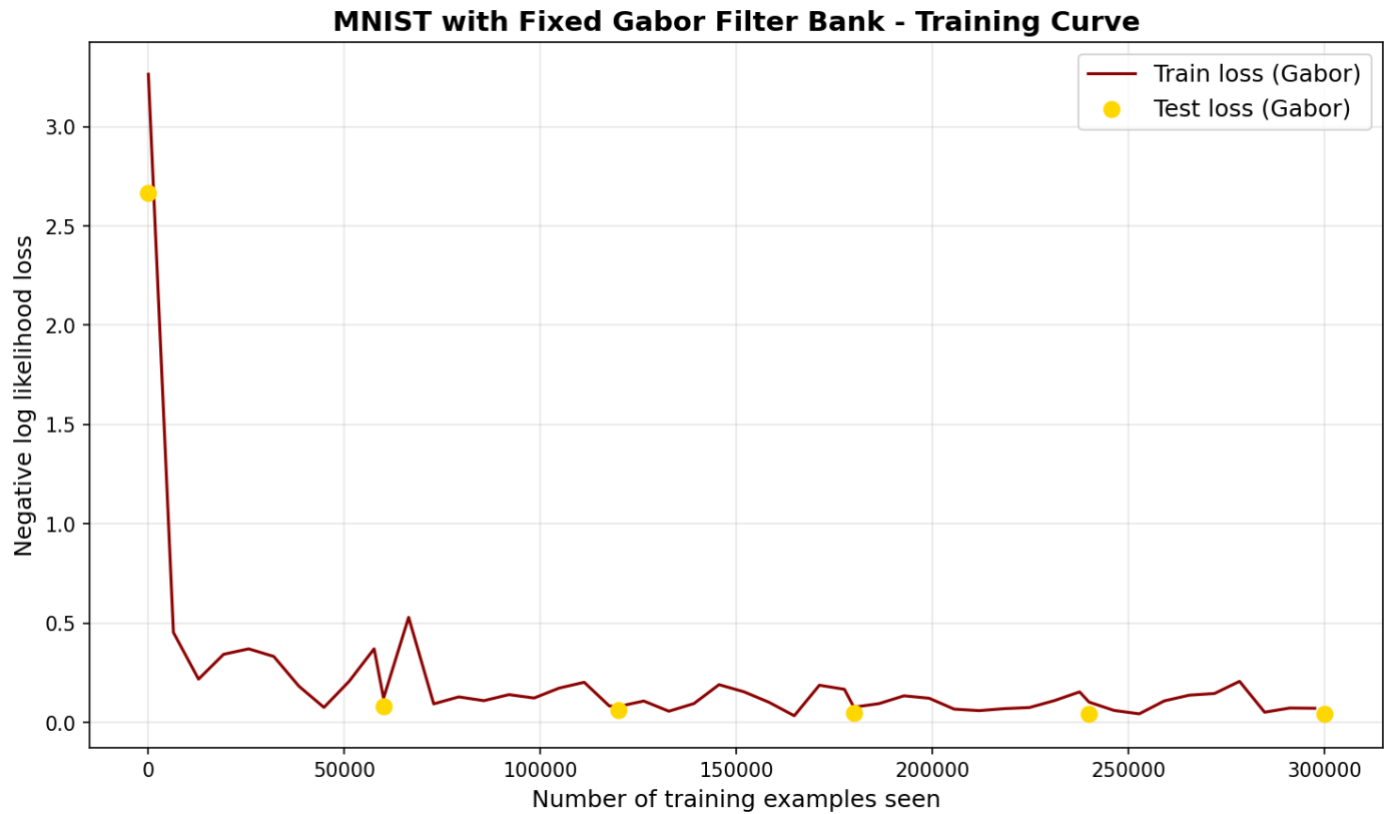
Gabor filters are mathematically derived from the product of a Gaussian envelope and a sinusoidal wave at a specific orientation and frequency. They are well studied in both neuroscience and computer vision because they closely resemble the receptive fields of simple cells in the mammalian visual cortex, the neurons responsible for detecting oriented edges and gratings in early visual processing. In other words, biological vision systems appear to use something very similar to Gabor filters as their first stage of processing, which makes them a natural candidate for the first layer of a digit recognition network.

Ten Gabor filters are created at evenly spaced orientations spanning 0 to 180 degrees, covering horizontal, vertical, and diagonal edge detection at multiple angles. Each filter is normalized to have zero mean and unit standard deviation before being loaded into the conv1 layer. The key difference from the standard network is that these weights are frozen immediately after being set, so backpropagation cannot modify them during training. The conv2 layer, dropout, and both fully connected layers remain trainable and learn entirely from the features that the fixed Gabor filters produce.



[Fig 18. Visualization of the 10 Gabor filters at different orientations shown as heatmaps]

Looking at the Gabor filter visualizations makes it immediately clear why these filters are useful for digit recognition. Each one is essentially a oriented stripe detector, with alternating bands of positive and negative weights aligned in a specific direction. When convolved with an image these filters produce strong positive responses along edges that match their orientation and suppress regions of uniform intensity. Together the 10 filters at different angles give the network sensitivity to edges and strokes in every direction, which is exactly the kind of information needed to distinguish between digits.



[Fig 19. Training curve for GaborNet showing convergence across 5 epochs]

The GaborNet trains successfully and reaches competitive accuracy with the standard learned network. This is a meaningful result because it suggests that the specific filter values learned during MNIST training are not uniquely optimal but rather that any reasonable set of edge detecting filters in the first layer is sufficient to give the deeper network what it needs. The learned filters and the Gabor filters both end up doing roughly the same job, detecting oriented edges and local contrast patterns, they just arrive at that result through completely different processes. One is derived from mathematical principles about optimal edge detection, the other emerges from thousands of gradient updates minimizing a classification loss.

The comparison also highlights something important about where the real learning happens in a convolutional network. The first layer filters, whether learned or fixed, are relatively simple feature detectors. The more complex and task-specific representations are built up in the deeper layers, which is why freezing the first layer and retraining everything else still produces a well-functioning network. The deeper layers adapt to work with whatever features the first layer provides.

```

● shamyas@Shamyas-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 extension1_gabor.py
=== Extension 1: Gabor Filter Bank as Fixed First Layer ===
GaborNet(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1), bias=False)
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (conv2_drop): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
Saved: outputs/extension_gabor_filters.png

Training GaborNet (conv1 frozen, rest trainable)...
Epoch 0 | Test Loss: 2.6684 | Acc: 11.43%
Epoch 1 | Test Loss: 0.0821 | Acc: 97.17%
Epoch 2 | Test Loss: 0.0607 | Acc: 97.99%
Epoch 3 | Test Loss: 0.0491 | Acc: 98.44%
Epoch 4 | Test Loss: 0.0439 | Acc: 98.46%
Epoch 5 | Test Loss: 0.0411 | Acc: 98.70%
Saved: outputs/extension_gabor_training.png

Extension 1 complete.
○ shamyas@Shamyas-MacBook-Air Project5_RecognitionusingDeepNetworks % █

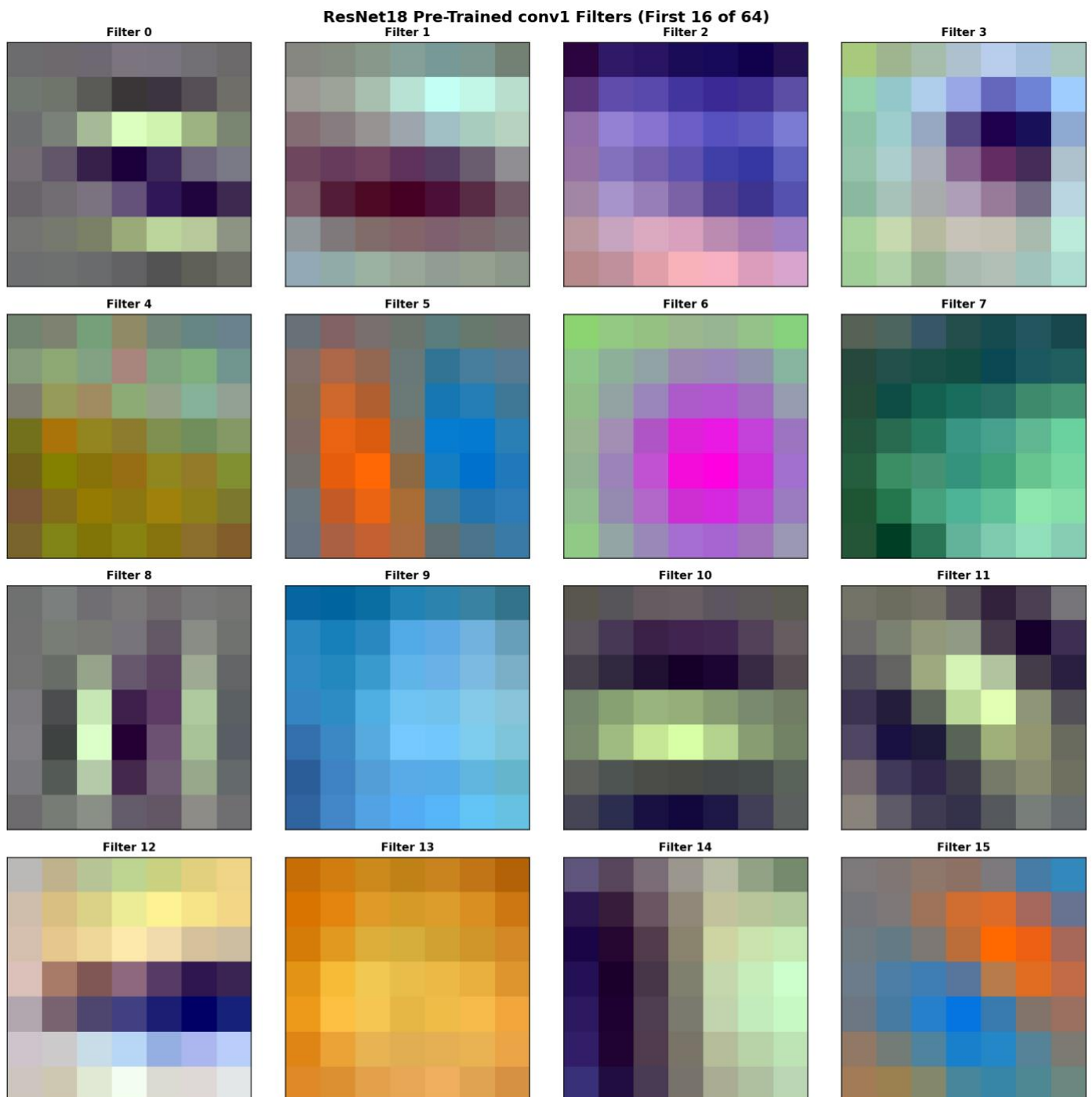
```

[Fig 20. Gabor Filter]

Extension 2: Pre-Trained ResNet18 Filter Analysis

The second extension takes the filter analysis from Task 2 and applies it to a much larger and more powerful network, a pre-trained ResNet18 loaded directly from PyTorch's model zoo. ResNet18 was trained on ImageNet, a dataset of over a million color photographs spanning 1000 object categories, and it represents a completely different scale of learning compared to the MNIST network built in this project. Examining its filters and comparing them to the ones learned by the simple digit classifier reveals how visual feature representations change as network depth, dataset size, and task complexity increase.

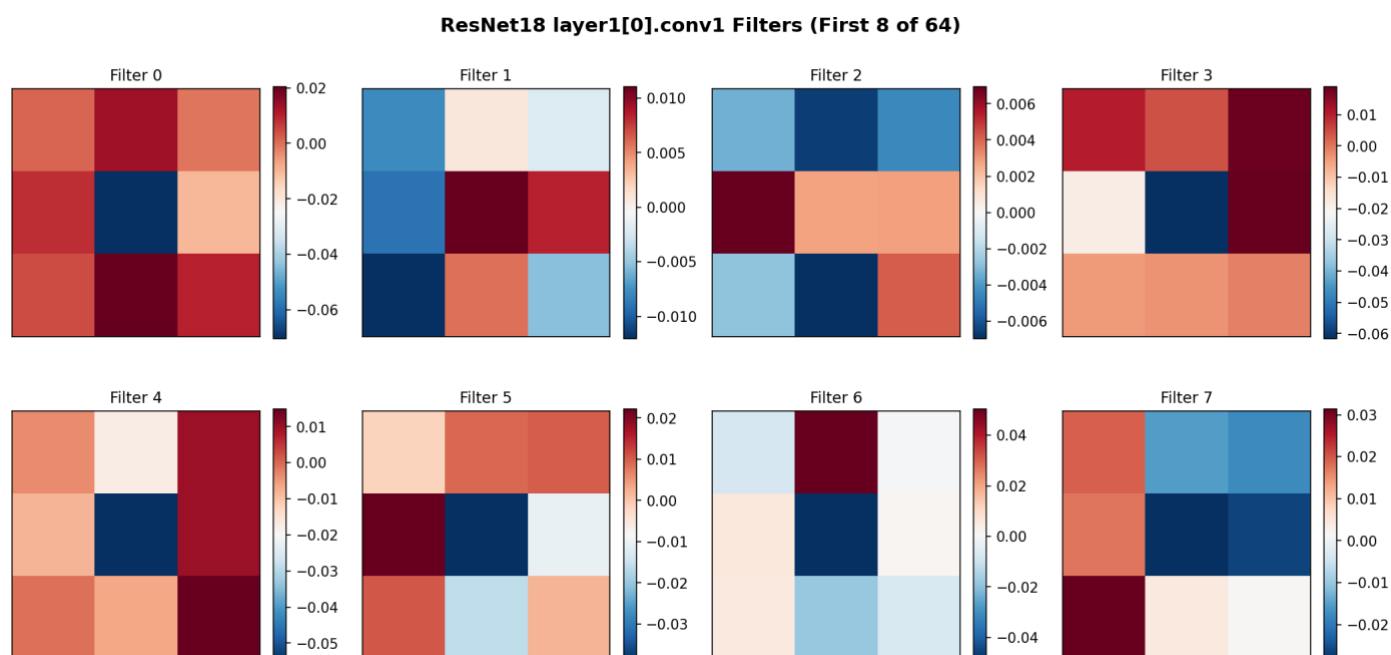
The first convolutional layer of ResNet18 has 64 filters of size 7 by 7 operating on 3 color channels, compared to the 10 filters of size 5 by 5 on 1 channel in the MNIST network. Each filter is therefore a 7 by 7 by 3 tensor, and visualizing it requires averaging across the three color channels or displaying each channel separately. The first 16 filters are visualized as normalized RGB images, with each filter's values rescaled to the 0 to 1 range for display.



[Fig 21. First 16 ResNet18 conv1 filters visualized as RGB heatmaps]

The ResNet18 filters look noticeably different from the MNIST filters. Where the MNIST conv1 filters had relatively simple patterns, some of the ResNet filters show more complex color opponent structures, responding differently to different color combinations rather than just to intensity edges. This makes sense given that ResNet18 was trained on color photographs where color information is meaningful and discriminative, whereas the MNIST network only ever saw grayscale images where the only relevant information is intensity.

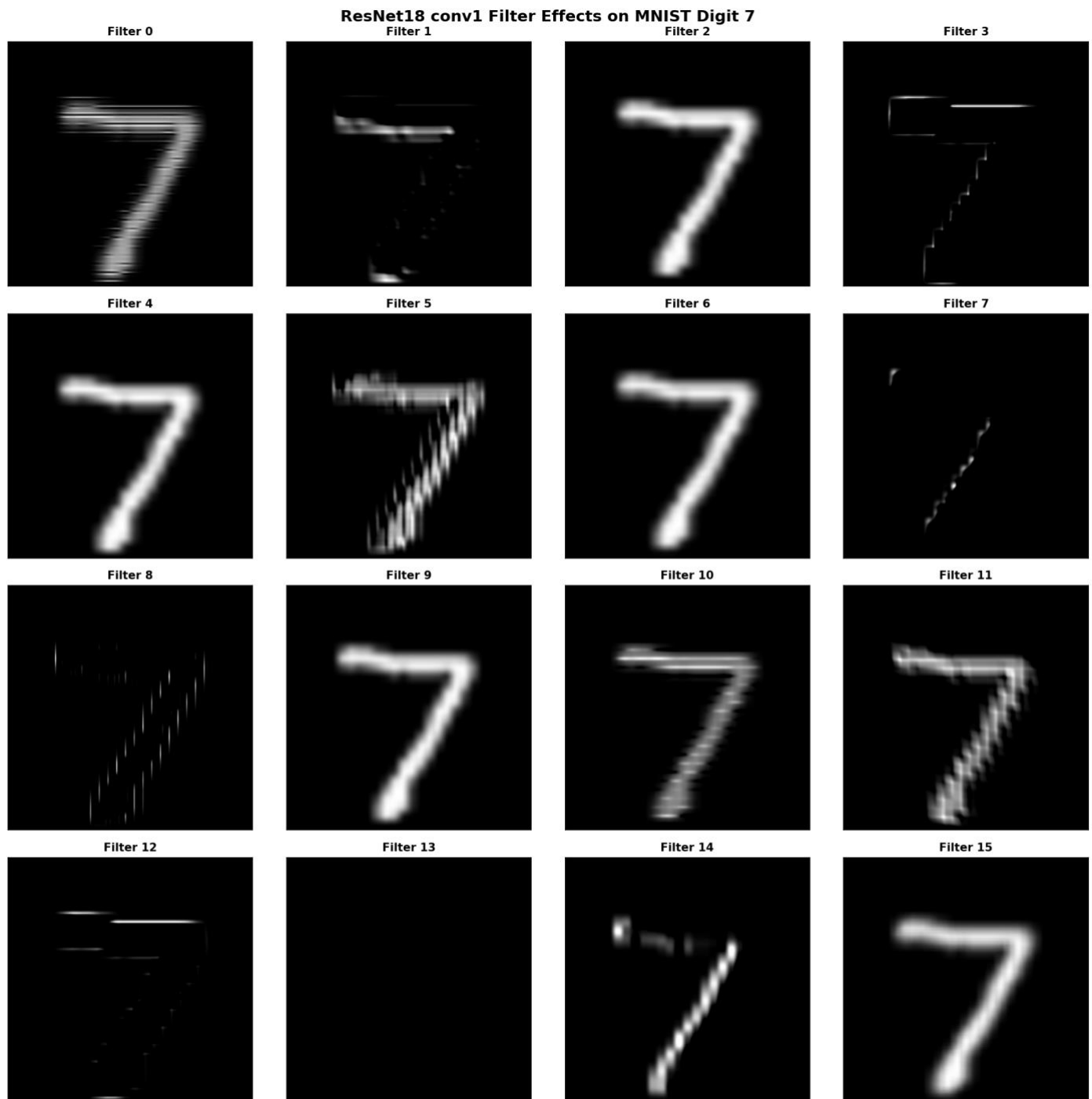
That said, many of the ResNet filters still show the same basic oriented edge structure seen in the MNIST filters, with elongated bands of alternating positive and negative values aligned in a specific direction. This suggests that oriented edge detection is a universal first step in visual processing regardless of the task, the dataset, or the network architecture. Whether you are recognizing handwritten digits or classifying photographs of 1000 different object categories, the first thing a convolutional network learns to do is detect edges.



[Fig 22. ResNet18 layer1 filters showing deeper representations beyond simple edge detection]

Looking at the filters from the first residual block one layer deeper reveals representations that are harder to interpret visually but clearly more complex than simple edge detectors. The patterns are less regular and less obviously interpretable, which is consistent with the general understanding that deeper layers in a convolutional network build increasingly abstract representations that combine the simple features detected in earlier layers into more complex structures. By the time you reach the deeper layers of ResNet18 the filters are responding to combinations of edges, textures, and shapes that correspond to object parts rather than to simple image gradients.

Applying the ResNet conv1 filters to an upscaled MNIST digit and visualizing the responses confirms that even filters trained on natural color photographs respond meaningfully to handwritten digit images. The edge detecting filters produce clear responses along the strokes of the digit, while the color opponent filters produce weaker and less structured responses since the input is grayscale. This cross-domain analysis demonstrates that low level visual features like edges are genuinely universal across tasks and datasets in a way that higher level features are not.



[Fig 22. ResNet18 conv1 filter effects applied to an upscaled MNIST digit]

```

shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks % python3.11 extension2_resnet_analysis.py
=== Extension 2: Pre-Trained ResNet18 Filter Analysis ===
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /Users/shamya/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100.0%
Loaded pre-trained ResNet18
conv1 weight shape: torch.Size([64, 3, 7, 7])
Saved: outputs/extension_resnet_filters.png
layer1[0].conv1 weight shape: torch.Size([64, 64, 3, 3])
Saved: outputs/extension_resnet_layer2_filters.png
Saved: outputs/extension_resnet_filter_effects.png

Extension 2 complete.
shamya@Shamya-MacBook-Air Project5_RecognitionusingDeepNetworks %

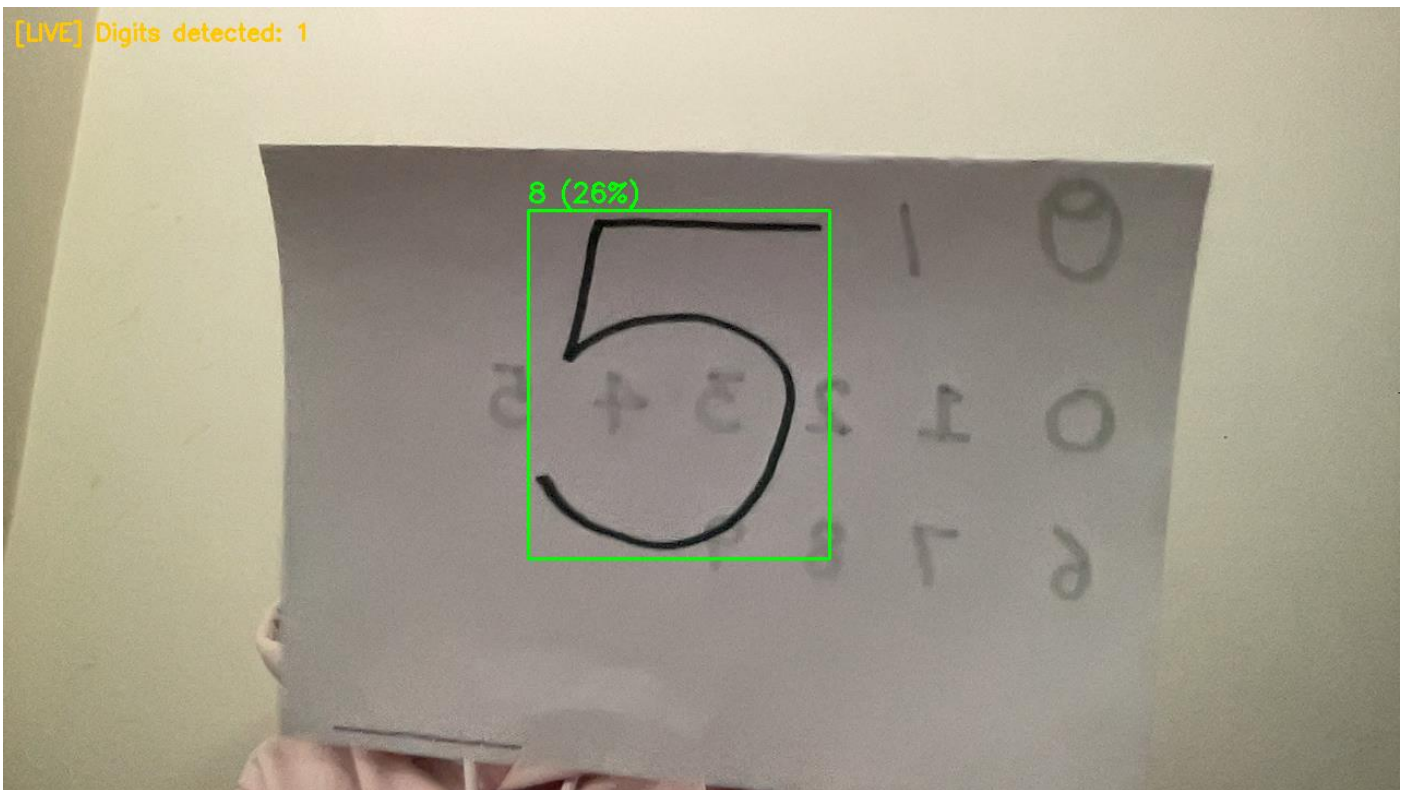
```

[Fig 23. Pre-Trained ResNet18 Filter Analysis]

Extension 3: Live Video Digit Recognition

The third extension takes the trained MNIST network out of the static evaluation context and deploys it as a real time application that processes live webcam video and attempts to classify handwritten digits as they appear in the frame. This felt like the most satisfying way to close out the project because it transforms what started as a dataset exercise into something that actually interacts with the physical world.

The application captures frames from the webcam continuously, converts each frame to grayscale, and runs a contour based detection algorithm to find regions of the image that might contain a digit. The detection works by applying Gaussian blurring to reduce noise, running Canny edge detection to find strong intensity transitions, dilating the edges to close small gaps in strokes, and then finding external contours in the resulting binary image. Each contour is filtered by its bounding box dimensions and aspect ratio to reject regions that are too small, too large, or too wide to plausibly be a single digit. Any region that passes these filters gets cropped, resized to 28 by 28 pixels, intensity inverted to match the MNIST format, normalized, and passed through the network for classification. The predicted digit and confidence score are drawn directly onto the live frame as a green bounding box with a label.



[Fig 24. Live webcam screenshot showing digit detection in action with green bounding boxes and predictions]

Testing the application revealed some interesting practical limitations that are worth discussing honestly. The contour based detection works reasonably well when a digit is isolated, clearly bounded, and takes up a moderate portion of the frame. It struggles with large digits whose strokes have gaps, because the detector sees multiple disconnected contour regions instead of one unified digit shape. It also picks up hands and paper edges as false positives when those regions happen to match the size and aspect ratio filters. In the screenshot above the detector found 3 regions and made predictions on two of them, labeling one as a 5 with 23% confidence and another as a 6 with 48% confidence, while the actual large 5 on the paper was too spread out across the frame to be captured as a single contour.

These limitations point to a real gap between how the network was trained and how it gets used in a live setting. The MNIST training data consists of tightly cropped, centered, cleanly isolated digits. A live video feed introduces variability in scale, position, lighting, background clutter, and partial occlusion that the

network never encountered during training. A more robust live application would use a sliding window approach or a dedicated object detector trained specifically for digit localization rather than relying on contour finding, but for the scope of this project the application successfully demonstrates the end to end pipeline from webcam input to neural network classification output running in real time.

REFLECTION

Coming into this project I had a decent theoretical understanding of how convolutional networks work but had never actually built one from scratch, trained it, and then systematically taken it apart to understand what was happening inside. The gap between reading about backpropagation and gradient descent and actually watching a network go from 11% to 98% accuracy in five minutes on your own laptop is larger than I expected. There is something that clicks when you see it happen in front of you that does not click from reading a textbook explanation.

The part of the project that surprised me most was Task 2, examining the filters. I expected the visualizations to look like noise or at best like vaguely structured patterns that I would have to squint at to interpret. Instead several of the filters were immediately recognizable as oriented edge detectors, almost identical in structure to the Gabor filters I later built by hand for Extension 1. The fact that gradient descent, starting from random initialization and optimizing purely for digit classification accuracy, independently arrived at something that mathematically trained engineers derived analytically decades ago was genuinely striking. It suggests that these kinds of edge detecting structures are not just one possible solution but something close to the optimal solution for early visual processing regardless of how you get there.

Transfer learning in Task 3 was another moment where the practical reality of something exceeded my expectation of it. Knowing theoretically that frozen convolutional features from one task can be reused for another is one thing. Watching a network trained on 60,000 digit images reach perfect accuracy on Greek letter classification using only 27 examples, in under 20 epochs, is another. The failure to generalize well to my own handwritten Greek symbols was equally instructive. With only 27 training examples drawn in one specific style the network had no way of knowing that alpha can look quite different depending on who writes it. That result alone says more about the importance of dataset diversity than any number of lectures on overfitting.

The transformer implementation in Task 4 forced me to think carefully about what convolutional networks are actually doing that transformers are not, and vice versa. Convolutions have spatial locality and translation equivariance baked in as architectural assumptions, which gives them a significant head start on image tasks. Transformers make no such assumptions and have to discover spatial relationships entirely from data through the attention mechanism. On MNIST both approaches converge to similar accuracy, but understanding why the CNN gets there faster in the early epochs and what the transformer is doing differently with its attention heads made the comparison genuinely educational rather than just a performance benchmark.

The hyperparameter experiment in Task 5 was humbling in a useful way. I went in expecting to find clear winners across each dimension and instead found that the baseline architecture was already close to optimal for this task. The differences between 5 filters and 32 filters, between 20 hidden nodes and 200, between ReLU and ELU, were all real but small. The lesson there is that for a well studied and relatively simple dataset like MNIST most reasonable architectural choices work, and the real challenge of deep learning lies in the harder problems where those choices start to matter much more.

If I were to continue this work the most interesting direction would be applying the same pipeline to a more challenging dataset where the architectural decisions have larger consequences. The Fashion MNIST dataset was mentioned in the assignment as a harder alternative and based on the experiment results I suspect the differences between filter counts and activation functions would be more pronounced there. Beyond that the live video application pointed toward a genuinely interesting engineering problem around robust digit localization in unconstrained scenes, which would require moving beyond contour detection toward a proper learned object detector.

Overall this project changed how I think about neural networks, not as black boxes that produce accurate predictions but as systems with interpretable internal structure that can be examined, modified, transferred, and deployed in ways that connect back to classical ideas in computer vision and signal processing. That

connection between the learned and the hand designed, between the modern and the classical, was the most valuable thing I took away from it.

ACKNOWLEDGEMENTS

The MNIST dataset used throughout this project was originally compiled by Yann LeCun, Corinna Cortes, and Christopher Burges and has served as a standard benchmark in the machine learning community for decades. The Greek letter dataset used for the transfer learning task was provided as part of the CS 5330 course materials.

The primary framework used for building and training all networks in this project was PyTorch, developed and maintained by Meta AI. The torchvision package provided both the MNIST dataset loader and the pre-trained ResNet18 weights used in Extension 2. OpenCV was used for image filtering operations in Task 2 and for the contour based detection pipeline in the live video application. Matplotlib was used for all visualizations and output plots throughout the project.

The PyTorch documentation and official tutorials were consulted during the initial network building phase, particularly for understanding the DataLoader API and the structure of a standard training loop. The assignment description and provided code snippets, including the GreekTransform class and the NetTransformer template, were used as starting points for Tasks 3 and 4 respectively.

Claude, an AI assistant made by Anthropic, was used during this project for help with code(minor), debugging and report writing guidance.

Professor's course materials and office hours were a resource throughout the project for clarifying requirements and discussing results.